

Limits of neighborhood-based CF

- Rating matrix is an overfit representation of user preferences
- Using the whole rating matrix for recommendation creates two problems
 - Efficiency (even with LSH)
 - Quality of recommendations may suffer

Example

- Liking “*Star Wars Ep. 5 DVD*” and “*Star Wars The Original Trilogy DVD set*” might be interpreted as separate signals, as few people might buy them both
- In reality, the two items have in common some feature (latent factor) that should capture their semantic relatedness (epic sci-fi sagas)

- Solution: dimensionality reduction
 - Extract a small set of latent dimensions that encode user’s taste
 - Smooth out noise to increase both quality and efficiency

Model-based CF

- Find a model of latent factors to map both users and items to
- Achieve that through Matrix Factorization
- Recommendation problem as rating prediction
- Optimization problem: find prediction of ratings with lowest error
 - Lowest RMSE on existing (user,item) ratings

Objective

```
from surprise import SVD
algo = SVD().fit(trainset)
```

$$R = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_n u_n v_n^T$$

$$\min_{U, V} \sum_{i,j} (r_{ij} - \hat{r}_{ij})^2 + \gamma \left(\sum_j \|u_j\|_2 + \sum_i \|v_i^T\|_2 \right)$$

Diagram illustrating the SVD decomposition of the rating matrix R (size $m \times n$).

The matrix R is decomposed into three matrices:

- U (size $m \times m$): Matrix of user latent factors $u_1, \dots, u_m$.
- Σ (size $m \times n$): Matrix of singular values $\sigma_1, \dots, \sigma_n$.
- V^T (size $n \times n$): Matrix of item latent factors $v_1^T, \dots, v_n^T$.

The decomposition is shown as:

$$R = U \Sigma V^T$$

Below the diagram, the update rules for the latent factors are given:

$$\Delta b_{ui} = \lambda \cdot (\varepsilon_{ui} - \gamma \cdot b_{ui})$$

$$\Delta u_{if} = \lambda \cdot (\varepsilon_{ui} \cdot v_{uf} - \gamma \cdot u_{if})$$

$$\Delta v_{uf} = \lambda \cdot (\varepsilon_{ui} \cdot u_{if} - \gamma \cdot v_{if})$$

Two popular decompositions for RecSys

- Singular Value Decomposition (SVD)
- Non-negative Matrix Factorization (NMF)

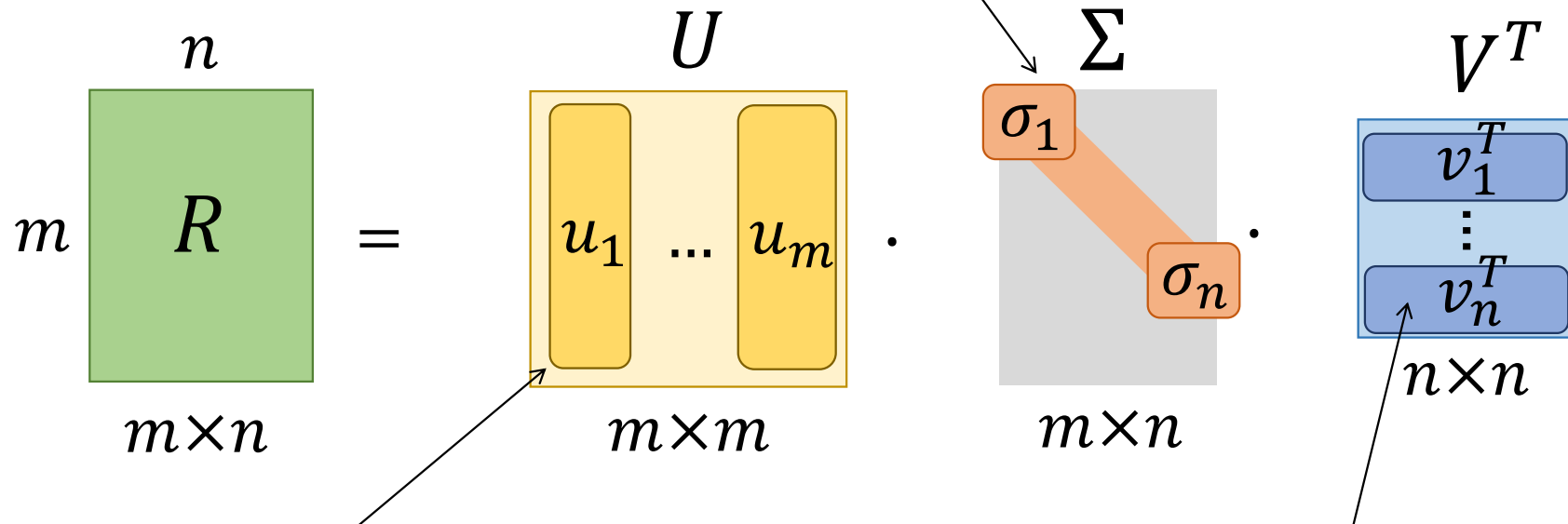
Singular Value Decomposition (SVD)

$$\begin{matrix} & n \\ & \boxed{R} \\ m & \\ & m \times n \end{matrix} = \begin{matrix} & m \\ & \boxed{U} \\ m & \\ & m \times m \end{matrix} \cdot \begin{matrix} & n \\ & \boxed{\Sigma} \\ m & \\ & m \times n \end{matrix} \cdot \begin{matrix} & n \\ & \boxed{V^T} \\ & n \times n \end{matrix}$$

- R is a matrix of real numbers (our rating matrix)
- A decomposition $R = U\Sigma V^T$ is guaranteed to exist, and it is unique
- U, V are orthogonal: square matrices such that $UU^T = \mathbb{I}$ and $VV^T = \mathbb{I}$ (where $\mathbb{I}$ is the identity matrix, a diagonal matrix of 1s)
- Σ is a diagonal matrix (all zeros but the values on the $n \times n$ diagonal) with values sorted in descending order of importance ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$)

(Notation varies, with these popular renaming: $\Sigma \rightarrow S$, $R \rightarrow M$, $U \rightarrow P$, $V \rightarrow Q$)

SVD interpretation



Left singular vectors

Information about the column space of R
Sorted in decreasing ability of explaining the variance of columns of R (i.e., variance of user ratings across items)

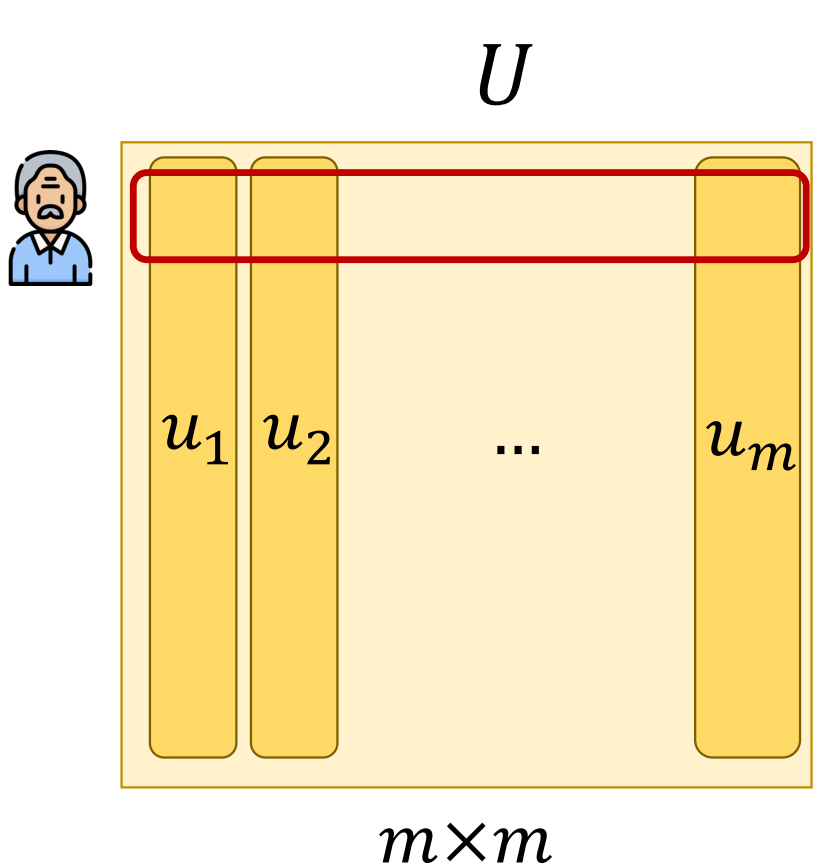
Singular values

Importance of singular vectors

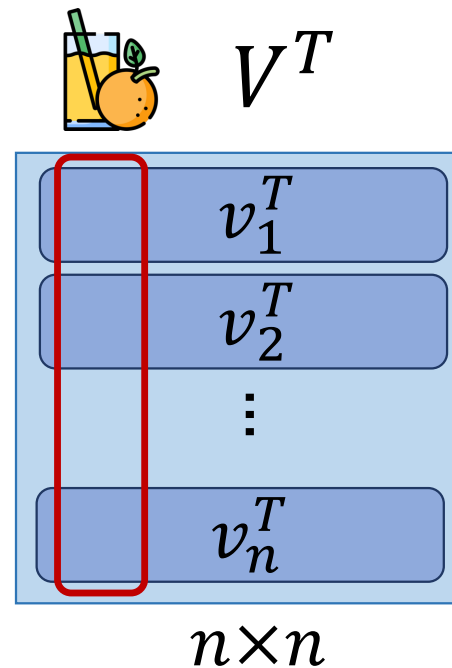
Right singular vectors

Information about the row space of R
Sorted in decreasing ability of explaining the variance of rows of R (i.e., variance of user ratings across users)

SVD interpretation



User-feature affinity matrix



Item-feature affinity matrix

- Both users and items are mapped over singular vectors

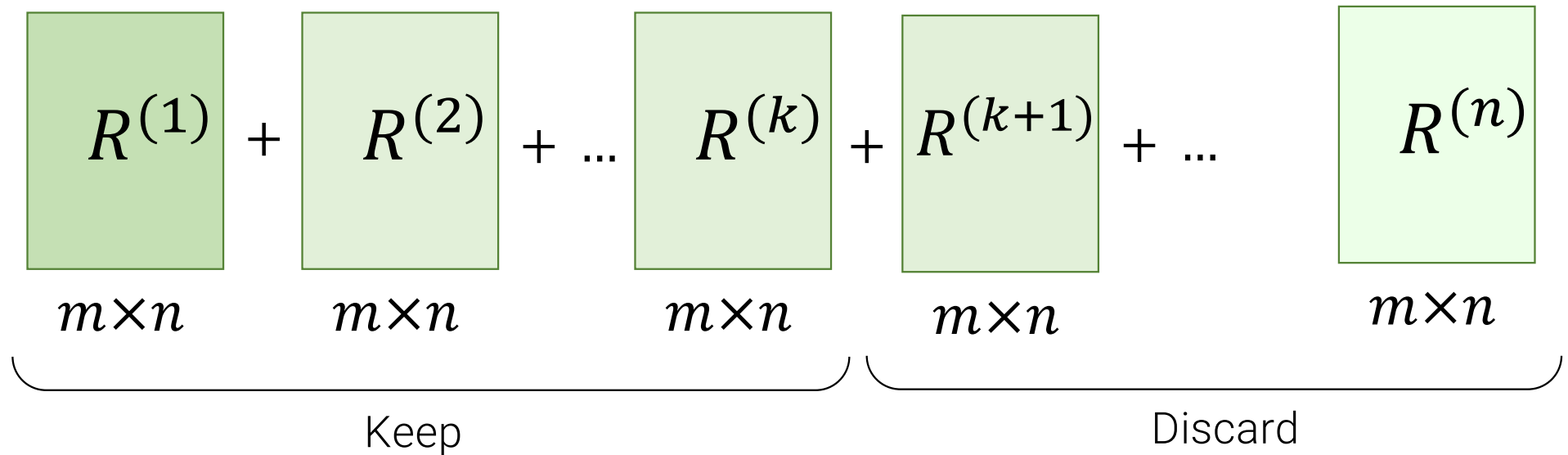
Singular Value Decomposition example

$$\begin{array}{c} R \\ \begin{array}{|c|c|c|} \hline 3 & 4 & 4 \\ \hline 2 & 5 & 1 \\ \hline 3 & 3 & 5 \\ \hline 2 & 3 & 4 \\ \hline \end{array} \end{array} = \begin{array}{c} R^{(1)} \\ \begin{array}{|c|c|c|} \hline 2.83 & 4.08 & 4.04 \\ \hline 2.08 & 2.99 & 2.97 \\ \hline 2.82 & 4.07 & 4.04 \\ \hline 2.35 & 3.39 & 3.36 \\ \hline \end{array} \end{array} + \begin{array}{c} R^{(2)} \\ \begin{array}{|c|c|c|} \hline 0.0 & -.02 & .02 \\ \hline -.06 & 2.00 & -1.98 \\ \hline .03 & -1.02 & 1.01 \\ \hline .02 & -.50 & .50 \\ \hline \end{array} \end{array} + \begin{array}{c} R^{(3)} \\ \begin{array}{|c|c|c|} \hline .17 & -.06 & -.06 \\ \hline -.02 & .01 & .01 \\ \hline .14 & -.05 & -.05 \\ \hline -.37 & .12 & .13 \\ \hline \end{array} \end{array}$$

- Information value decreases as j in $R^{(j)}$ increases
- Keep only the first matrices to approximate R and save space/time

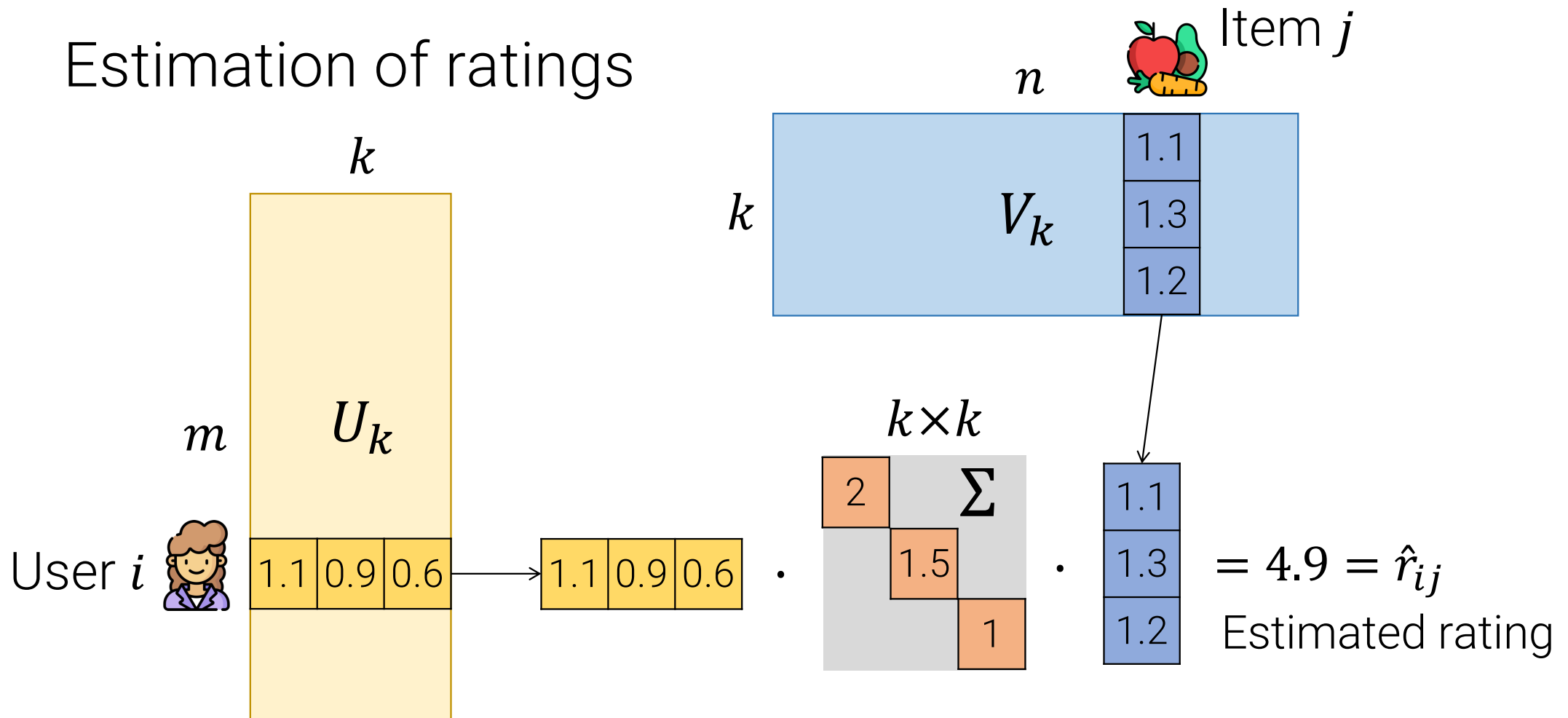
SVD interpretation

$$R = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_n u_n v_n^T$$



- The first k matrices carry most of the information
- Discard matrices from $k + 1$ to n
- Cutoff at k is guaranteed to be the best rank- k approximation

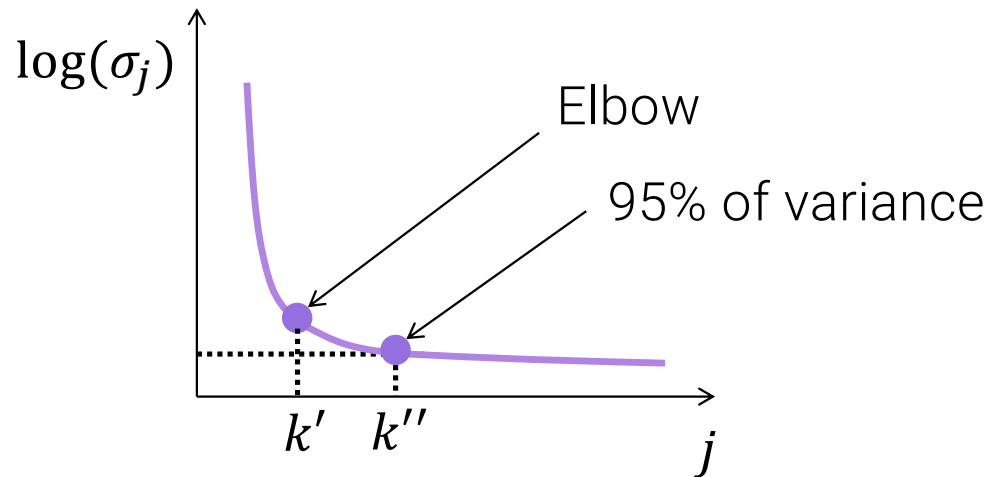
Estimation of ratings



Both users and items are in the same feature space

How to find a good k ?

1) Elbow method



2) Downstream task optimization

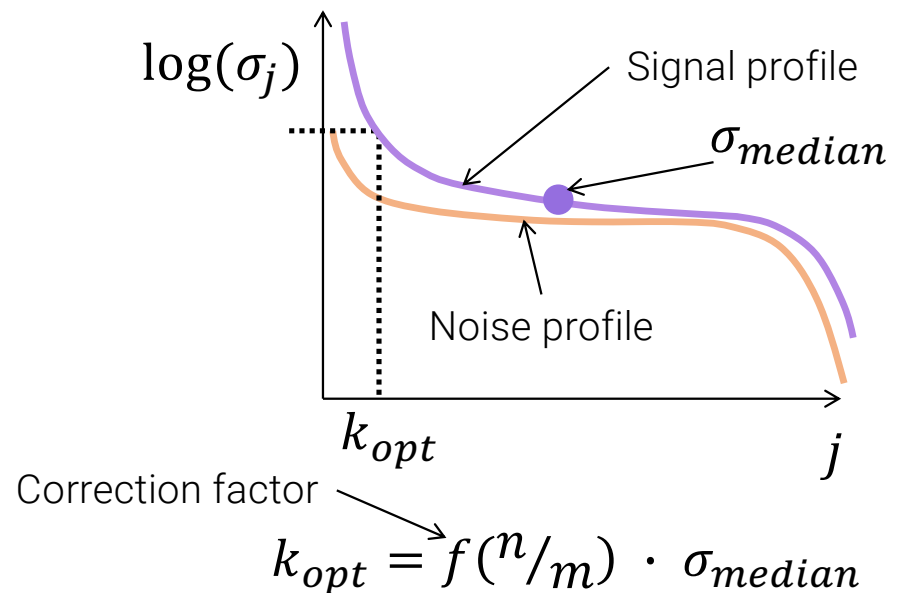
Try a bunch of values of k in the validation set, and select the one that minimizes the recommendation error

Gaivsh, Donoho "The Optimal Hard Threshold for Singular Values is $4/\sqrt{3}$ ", 2013
<https://arxiv.org/pdf/1305.5870.pdf>

3) Optimal threshold

Assumption: mean=0, variance=1

$$R = R_{\text{signal}} + \gamma R_{\text{noise}}$$



SVD in Python

```
import numpy as np
from numpy.linalg import svd
from optht import optht # package for optimal thresholding

R = np.array([[3,4,4],[2,5,1],[3,3,5],[2,3,4]])

U, S, V = svd(R)

# get optimal k
k = optht(R, sv=S)

# estimate R with a
R_est = (U[:, range(k)] * S[range(k)]).dot(V[range(k),:])
```

<https://github.com/erichson/optht>

Funk SVD

- Update values in matrices R_b , U_k and V_k^T
 - $\Delta b_{ij} = \lambda \cdot (\varepsilon_{ij} - \gamma \cdot b_{ij})$
 - For each feature f :
 - $\Delta u_{if} = \lambda \cdot (\varepsilon_{ij} \cdot v_{jf} - \gamma \cdot u_{if})$
 - $\Delta v_{jf} = \lambda \cdot (\varepsilon_{ij} \cdot u_{if} - \gamma \cdot v_{jf})$
- Repeat for every datapoint
- Loop on all datapoints for #epochs
- Parameters
 - λ : learning rate (0.005)
 - γ : regularization term (0.02)

NOT SVD ANYMORE

Simon Funk "Netflix update, try this at home", 2006
<https://sifter.org/~simon/journal/20061211.html>

Funk SVD in Python

```
from surprise import SVD
from surprise import accuracy
from surprise.model_selection import train_test_split

trainset, testset = train_test_split(data, test_size=.25)

algo = SVD(n_factors=10, n_epochs=10, lr_all=0.001, reg_all=0.01)

algo.fit(trainset)
predictions = algo.test(testset)

accuracy.rmse(predictions)
```

Non-negative Matrix Factorization (NMF)

$$\begin{matrix} & k \\ m & U \\ & m \times k \end{matrix} \cdot \begin{matrix} & n \\ k & V \\ & k \times n \end{matrix} = \begin{matrix} & & \\ & \hat{R} \\ & m \times n \end{matrix} \approx \begin{matrix} & & \\ & R \\ & m \times n \end{matrix}$$

- All matrices must contain only **non-negative** values
- U matrix of basis vectors, one per column
- V matrix of activation/weight/gain vectors, one per row
- Somewhat similar to approximated SVD
- Main advantage: interpretability
 - Values in U and V can be directly interpreted as preference weights on latent factors

Solving NMF

- Minimize distance D
 - Euclidean: $\sum_{i,j} (r_{ij} - \hat{r}_{ij})^2$
 - Kullback-Leibler divergence: $\sum_{i,j} \left(r_{ij} \cdot \log \frac{r_{ij}}{\hat{r}_{ij}} - r_{ij} + \hat{r}_{ij} \right)$
- Find matrices U, V such that D is minimized
- Use gradient descent, like for approximated SVD

```
from surprise import NMF
#learning rates and reg terms are per matrix
algo = NMF(n_factors=10, n_epochs=10, ...)
```


SVD++

- Extension of Funk SVD to incorporate implicit feedback data (user clicks, number of song plays, ...)
- Adds a latent factor to account that a user rated an item, regardless of score

$$\hat{r}_{ij} = \underset{\text{User bias}}{b_i} + \underset{\text{Item bias}}{b_j} + v_j^T \cdot \left(u_i + \frac{1}{\sqrt{|R_i|}} \sum_{l \in R_i} y_l \right)$$

New latent factor

Set of items rated by user i

Implicit ratings

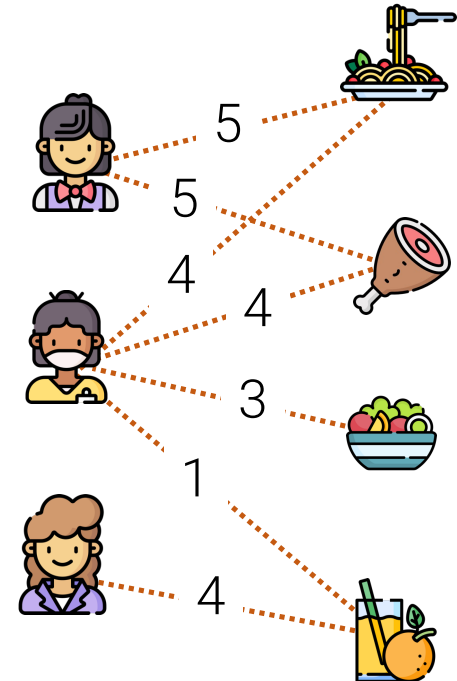
			
		1	1
		1	1
	1	1	
		1	1

```
from surprise import SVDpp
trainset, testset = train_test_split(data, test_size=.25)
algo = SVDpp(n_factors=10, n_epochs=10, lr_all=0.001, reg_all=0.01)
```

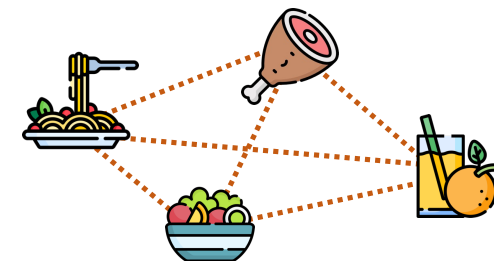
Recommendation problem cast into network

- Link analysis and recommendation models are intimately related
- Two different paradigms to model the same underlying relationships

				
	5	5		
	4	4	3	1
				4



User-projection



Item-projection

Assignment

- Using MovieLens (train/test split), compare the RMSE of:
- Popularity-based approach with damping/recency ranking (non-personalized)
- Content-based approach (on tags or genres)
- Neighborhood-based collaborative filtering
- Model-based collaborative filtering (SVD, NMF)